

QUALITY ASSURANCE TEST REPORT

Vishal Gold — Android Mobile Application

Project	Vishal Gold — B2B/B2C Jewelry App
Prepared For	Vishal Gold Management & Technical Stakeholders
Prepared By	QA Engineering Team
Version	v1.2 — Final Release
Date	February 25, 2026

Document Control

Attribute	Detail
Document Title	Quality Assurance Test Report — Final Release
Application Name	Vishal Gold — B2B/B2C Jewelry Mobile Application
Platform	Android (API 21+)
Document Version	v1.2 — Final
Classification	CONFIDENTIAL — Client Delivery
Test Cycle	Cycle 1 (Pre-release) + Cycle 2 (Post-remediation retest)
Test Period	February 17 – February 25, 2026
Report Date	February 25, 2026
Prepared By	QA Engineering Team
Reviewed By	Engineering Team Lead
Distribution	Vishal Gold CTO, Project Manager, Development Lead

Revision History

Version	Date	Author	Change Summary
v1.0	Feb 19, 2026	QA Team	Initial test report — Cycle 1 results
v1.1	Feb 22, 2026	QA Team	Updated with Cycle 2 retest results and defect closures
v1.2	Feb 25, 2026	QA Team	Final release — all sections complete, sign-off added

Table of Contents

1. Executive Summary
2. Test Objectives and Scope
3. Test Environment Details
4. Test Strategy and Methodology
5. Test Coverage Summary
6. Test Execution Results
7. Defect Summary
8. Detailed Defect Descriptions
9. Regression Testing Results
10. Risks and Quality Assessment
11. Recommendations and Next Steps
12. Final Quality Status and Sign-Off
13. Appendix A — Test Data Reference
14. Appendix B — Tools and Frameworks
15. Appendix C — Test Metrics Summary

1. Executive Summary

This report presents the comprehensive Quality Assurance findings for the Vishal Gold Android mobile application, a B2B and B2C commerce platform designed to serve Gold jewellery wholesalers, retailers, and administrators. QA testing was conducted across two formal test cycles. Cycle 1 constituted the primary functional, regression, and exploratory test pass against the feature-complete pre-release build. Cycle 2 was a targeted regression and defect validation cycle performed after developer remediation of all Cycle 1 defects.

A total of 1,247 test cases were executed across 18 functional areas spanning all three user roles — Retailer, Wholesaler, and Administrator. Testing was performed on three physical Android devices and two emulators covering Android versions 10 through 14, representing the production target device range. Supplementary automated test suites were developed and executed using Flutter's integration test framework.

Cycle 1 identified 43 defects: 2 Critical, 8 High, 18 Medium, 11 Low, and 4 Cosmetic. All 43 defects were assigned to the development team and resolved prior to Cycle 2. Following Cycle 2 regression testing, 40 of 43 defects were confirmed closed with no reopen. Three Low-severity defects were accepted by the project team as Known Limitations documented in the Known Issues Log. No new defects were introduced during remediation.

The final quality verdict for this release is PASS with conditions. The application meets all functional acceptance criteria defined in the project requirements. All Critical and High defects have been resolved. The application is recommended for production deployment subject to the deployment of Firestore security rules as identified in the VAPT assessment and detailed in the companion Security Disclosure Document.

Quality Metrics Dashboard

Metric	Cycle 1	Cycle 2 (Final)	Target
Total Test Cases	1,247	1,247	≥ 1,200
Test Cases Passed	1,085	1,236	≥ 1,200
Test Cases Failed	162	11	0 Critical/High
Test Cases Blocked	38	0	0
Test Cases Not Applicable	10	10	N/A
Pass Rate	87.0%	99.1%	≥ 98%
Defects Raised	43	0	0 new in Cycle 2
Defects Closed	0	40	100% Critical/High
Defects Open	43	3 (Low/Known)	0 Critical/High

Automation Coverage	—	68%	≥ 60%
---------------------	---	-----	-------

VERDICT: PASS with Conditions — all Critical and High defects resolved. Three Low defects accepted as Known Limitations. Application approved for production deployment.

2. Test Objectives and Scope

2.1 Test Objectives

The primary objectives of the QA test programme were to verify that all features specified in the project requirements document function correctly across the supported device range; to validate the role-based access control model by testing all three user personas (Retailer, Wholesaler, Administrator); to identify and document defects prior to production deployment; to ensure the application performs acceptably under target load conditions; and to validate the application's accessibility and usability against the defined quality acceptance criteria.

2.2 In-Scope Features

Module	Sub-Features Tested
Authentication	OTP login, email login, role assignment, session persistence, logout
User Onboarding	Registration flow, role selection, profile setup, field validation
Home & Navigation	Featured banners, bottom navigation, role-specific content rendering
Product Catalog	Category grid, subcategory browsing, list and grid views, search, filters
Product Detail	Image gallery, specifications table, description, wishlist, add-to-cart
Cart Management	Add/remove items, quantity update, price calculation, order placement
Sample Orders	Sample request flow (Wholesaler role), quantity limits, confirmation
Wishlist	Add/remove from wishlist, persistent across sessions
Order History	Order list, order detail view, status tracking
Push Notifications	FCM token registration, notification delivery, deep links
Admin — Products	Add, edit, delete products, image upload, subcategory assignment
Admin — Banners	Upload, reorder, activate/deactivate banners

Admin — Staging	Stage changes, preview, publish to live product collection
Admin — Orders	View all orders, filter by status, update order status
Admin — Notifications	Compose and send broadcast push notifications
Profile	View and edit profile fields, role display
Offline / Connectivity	Offline error handling, reconnection behaviour
Deep Link Routing	Push notification deep links navigate to correct screens

2.3 Out-of-Scope Items

The following items were outside the scope of this test cycle and are documented for completeness: iOS platform (not in project scope); web or browser-based access to the application; backend Firebase infrastructure performance testing beyond application-level load testing; Google Play Store listing content review; and payment gateway integration testing (no payment features are included in the current version).

3. Test Environment Details

3.1 Application Under Test

Parameter	Value
Application	Vishal Gold
Target Platform	Android (minSdkVersion 21 — Android 5.0+)
Framework	Flutter 3.32 / Dart 3.9
Backend	Google Firebase (Auth, Firestore, Storage, FCM)
Test Build Type	Debug (Cycle 1) + Release (Cycle 2 regression)
Package Name	com.vishalgold.app

3.2 Test Devices

Device	OS Version	API Level	Type	Purpose
Samsung Galaxy S23	Android 14	34	Physical	Primary — flagship test
Google Pixel 6a	Android 13	33	Physical	Secondary — baseline test
Samsung Galaxy A52	Android 12	31	Physical	Mid-range device test
Android Emulator x86_64	Android 10	29	Emulator	Low API regression
Android Emulator ARM	Android 11	30	Emulator	Alternate arch validation

4. Test Strategy and Methodology

The QA test programme was structured around a risk-based testing approach, prioritising test effort on the features with the highest business criticality and defect probability. Authentication, role-based access control, and financial data integrity (cart pricing and order placement) were assigned the highest risk weighting and received the most intensive test coverage. The test programme comprised five distinct test types:

Functional Testing

Systematic test case execution against each feature in the scope list, verifying that every specified behaviour produces the expected output under both positive (happy path) and negative (boundary and error) conditions. Test cases were derived from the project requirements document and supplemented by tester-identified exploratory scenarios.

Role-Based Access Control (RBAC) Testing

Dedicated test scenarios were designed to verify that the Retailer, Wholesaler, and Administrator personas each see only the screens, features, and data appropriate to their role. Cross-role access attempts — including navigating to admin URLs with a non-admin session — were explicitly tested.

Regression Testing

A full regression suite of 847 automation-eligible test cases was implemented using Flutter's `integration_test` framework and executed on both debug and release APK builds. The regression suite was run in full at the start of Cycle 2 to detect any regressions introduced by developer remediation of Cycle 1 defects.

Exploratory Testing

Unscripted exploratory testing sessions were conducted across all application modules using session-based test management. Exploratory sessions were time-boxed to 90 minutes each, with a written mission statement and findings log. This technique identified several of the higher-severity defects not covered by the scripted test case library.

Performance and Load Testing

Application startup time, screen-to-screen navigation latency, and Firestore read latency were measured across all test devices. Cold start and warm start times were

measured and compared against the acceptance criteria defined as: cold start ≤ 3 seconds on mid-range device, screen transition $\leq 300\text{ms}$.

4.1 Entry and Exit Criteria

Criterion Type	Condition
Entry — Cycle 1	Feature-complete build provided by development; no unresolved build blockers
Entry — Cycle 2	Developer confirmation of all Cycle 1 defect remediations; new APK build available
Exit — Cycle 1	All planned test cases executed; all Critical/High defects documented
Exit — Cycle 2	Pass rate $\geq 98\%$; 0 open Critical defects; 0 open High defects; defects not meeting closure criteria documented
Suspend Criteria	More than 25% of test cases blocked by a single critical defect — resume on fix

5. Test Coverage Summary

Module	Test Cases	Automated	Manual	Pass Rate (Cycle 2)
Authentication	98	72	26	100%
User Onboarding	54	32	22	100%
Home & Banners	47	28	19	100%
Product Catalog	156	98	58	100%
Product Detail	72	44	28	98.6%
Cart Management	89	61	28	100%
Sample Orders	63	35	28	100%
Wishlist	38	28	10	100%
Order History	55	34	21	100%
Push Notifications	48	22	26	95.8%
Admin — Products	112	68	44	100%
Admin — Banners	44	28	16	100%
Admin — Staging/Preview	67	38	29	98.5%
Admin — Orders	58	36	22	100%
Admin — Notifications	42	24	18	100%
Profile	38	20	18	100%
Offline Behaviour	34	12	22	97.1%
Deep Link Routing	32	16	16	100%
TOTALS	1,247	808	439	99.1%

6. Test Execution Results

6.1 Overall Execution Statistics

Status	Cycle 1 Count	Cycle 2 Count	Change
Pass	1,085	1,236	+151
Fail	162	11	-151
Blocked	38	0	-38
Not Applicable	10	10	0
Total Executed	1,285	1,247	
Pass Rate	87.0%	99.1%	+12.1%

The 10 test cases marked Not Applicable in Cycle 2 relate to features that were descoped from the production release by mutual agreement with the project management team: specifically, the in-app payment integration (deferred to Phase 2) and the social sharing features (postponed pending legal review of sharing policy).

6.2 Defect Discovery Rate by Cycle

Severity	Cycle 1 Found	Cycle 2 New	Total	Open at Report Date
Critical	2	0	2	0
High	8	0	8	0
Medium	18	0	18	0
Low	11	0	11	3 (Known Limitations)
Cosmetic	4	0	4	0
TOTAL	43	0	43	3

7. Defect Summary

7.1 Defect Status Overview

All 43 defects raised during Cycle 1 were reviewed, prioritised, and assigned to the development team within 24 hours of discovery. The development team resolved all Critical and High defects within 48 hours. Medium defects were resolved within the Cycle 1 to Cycle 2 remediation window. Low and Cosmetic defects were triaged individually; three Low-severity items were accepted as Known Limitations by the project team. Zero Critical or High defects remain open at the time of this report.

Severity	Raised	Resolved	Accepted (Known)	Open
Critical	2	2	0	0
High	8	8	0	0
Medium	18	18	0	0
Low	11	8	3	0 (3 accepted)
Cosmetic	4	4	0	0
TOTAL	43	40	3	0

7.2 Defect Distribution by Module

Module	Critical	High	Medium	Low	Cosmetic	Total
Authentication	1	2	3	1	0	7
Product Catalog	0	1	4	2	1	8
Cart Management	1	1	2	1	0	5
Admin — Products	0	2	4	2	1	9
Admin — Staging/Preview	0	1	2	1	1	5
Push Notifications	0	1	2	2	0	5
Offline Behaviour	0	0	1	2	0	3
Product Detail	0	0	0	0	1	1
Others	0	0	0	0	0	0
TOTAL	2	8	18	11	4	43

8. Detailed Defect Descriptions

This section documents all 43 defects raised during the test programme. Each entry includes the defect identifier, severity, module, a complete description of the observed and expected behaviour, steps to reproduce, and the resolution applied. The first twelve defects are described in full detail. All remaining defects are summarised in the consolidated table in Section 7.2. Full defect records are available in the project defect tracking system.

QA-001 [CRITICAL] — Order Subtotal Incorrect When Item Quantity Exceeds 99

Attribute	Detail
Defect ID	QA-001

Severity	CRITICAL
Status	CLOSED
Module	Cart — Order Placement

Description

When a user added more than 99 units of the same cart item and proceeded to checkout, the subtotal calculation in the `CartProvider.calculateSubtotal()` method returned an incorrect value. Analysis identified an integer overflow in the quantity multiplication when the result exceeded the maximum value of a 32-bit integer, caused by the use of a non-null `int` type without bounds checking.

Steps to Reproduce

1. Add any product to cart. 2. Update quantity to 9999. 3. Navigate to Cart screen. 4. Observe: Subtotal displays as a negative or truncated value.

Expected Behaviour

Subtotal correctly reflects unit price multiplied by quantity for all valid quantities.

Resolution Applied

Quantity input field capped at 999. Calculation migrated to double with explicit precision handling.

QA-002 [CRITICAL] — OTP Session State Persists After Explicit Logout — Re-login Bypassed

Attribute	Detail
Defect ID	QA-002
Severity	CRITICAL
Status	CLOSED
Module	Authentication

Description

A defect was identified where a successfully logged-in session was not fully invalidated by the logout action. The Firebase Auth `currentUser` object remained non-null in memory after calling `FirebaseAuth.instance.signOut()` because the `signOut` operation was not awaited. Subsequent navigation to the home screen could occur before the `signOut` future completed, leaving the previous user session active.

Steps to Reproduce

1. Log in as a Retailer. 2. Navigate to Profile. 3. Tap Logout. 4. Immediately navigate to the Home screen via the back gesture. 5. Observe: User remains authenticated without re-entering credentials.

Expected Behaviour

Logout fully invalidates the session before any navigation away from the logout confirmation screen.

Resolution Applied

await keyword added to the signOut() call. Navigation is blocked until signOut() future resolves.

QA-003 [HIGH] — Product Image Upload Fails Silently on Files Exceeding 3 MB

Attribute	Detail
Defect ID	QA-003
Severity	HIGH
Status	CLOSED
Module	Admin — Products

Description

When an administrator attempted to upload a product image exceeding 3 MB in file size, the upload progress indicator would animate briefly and then disappear, leaving the product image field empty and displaying no error message. The underlying Firebase Storage upload task was failing with a quota exceeded response, but the exception was caught and swallowed in the image upload handler without user notification.

Steps to Reproduce

1. Log in as Administrator. 2. Navigate to Product Management > Add Product. 3. Select an image file larger than 3 MB. 4. Observe: No error message displayed; product saved without image.

Expected Behaviour

A clear error message is displayed: 'Image too large. Please select an image under 3 MB.'

Resolution Applied

File size validation added pre-upload. User-facing error SnackBar implemented for files exceeding 3 MB. Success and error states are now correctly communicated.

QA-004 [HIGH] — Wholesaler-Role User Can Access Retailer-Only Pricing on Product Detail

Attribute	Detail
Defect ID	QA-004
Severity	HIGH
Status	CLOSED
Module	Authentication

Description

The product detail screen displayed retail pricing to Wholesaler-role users under a specific navigation path. When a Wholesaler navigated to a product detail via the Search Results screen (as opposed to the Category Grid), the role check in the ProductDetailScreen widget read the role from the widget's initialisation arguments rather than from the live AuthProvider, which defaulted to the Retailer role if no role was passed in the route arguments.

Steps to Reproduce

1. Log in as a Wholesaler. 2. Use the search bar to find a product. 3. Tap the product from search results. 4. Observe: Retail pricing displayed instead of wholesale pricing.

Expected Behaviour

Wholesale pricing consistently displayed to Wholesaler-role users regardless of navigation source.

Resolution Applied

Role determination in ProductDetailScreen refactored to always read from Provider.of<AuthProvider>, eliminating route argument dependency.

QA-005 [HIGH] — Staging Preview Reflects Stale Data — Published Changes Not Shown

Attribute	Detail
Defect ID	QA-005
Severity	HIGH
Status	CLOSED
Module	Admin — Staging

Description

After an administrator published a staged product batch, the live product catalog was correctly updated. However, the Admin Preview screen continued to display the now-

published staging data rather than refreshing to show the current live product state. The preview cache was not invalidated on publish, meaning the administrator's preview remained in a permanently stale state until the application was fully restarted.

Steps to Reproduce

1. Log in as Administrator. 2. Stage product changes. 3. Preview staged changes. 4. Publish staged changes. 5. Navigate back to the preview screen. 6. Observe: Preview continues to show staged (now-published) state, not live state.

Expected Behaviour

Preview screen reloads live product data after a publish operation.

Resolution Applied

PreviewProvider.clearPreview() called after successful publish. Preview screen forces a live data reload on next navigation.

QA-006 [HIGH] — Deep Link from Push Notification Navigates to Incorrect Product

Attribute	Detail
Defect ID	QA-006
Severity	HIGH
Status	CLOSED
Module	Push Notifications

Description

Push notifications containing product deep links correctly opened the application, but navigated to the wrong product detail screen. An off-by-one indexing error in the deep link resolver caused the resolver to retrieve the product at index $n-1$ instead of the intended product at the specified product ID. This was reproducible for all product-specific push notifications.

Steps to Reproduce

1. Send a push notification targeting '{{productId: abc123}}'. 2. Tap the notification. 3. Observe: Application opens to 'abc122' product detail instead of 'abc123'.

Expected Behaviour

Deep link navigates to the exact product specified in the notification payload.

Resolution Applied

Deep link routing logic corrected to use Firestore document ID lookup rather than collection index.

QA-007 [HIGH] — Subcategory Deletion Orphans Child Products — No Cascade Warning

Attribute	Detail
Defect ID	QA-007
Severity	HIGH
Status	CLOSED
Module	Admin — Products

Description

When an administrator deleted a subcategory that contained active products, the products remained in Firestore with their subcategoryId references intact. As the referenced subcategory no longer existed, the products became orphaned and were no longer displayed in any category or subcategory listing. The products remained in Firestore but were effectively inaccessible to users.

Steps to Reproduce

1. Log in as Administrator. 2. Create a subcategory with at least one product. 3. Navigate to Subcategory Management. 4. Delete the subcategory without first reassigning or deleting the products. 5. Observe: Products are missing from all catalog views.

Expected Behaviour

Deletion is blocked if child products exist, or the administrator is presented with an explicit warning and the option to reassign products before deletion.

Resolution Applied

Deletion blocked if `childProducts.count > 0`. Warning dialog displayed listing affected products with reassign option.

QA-008 [HIGH] — Concurrent Cart Writes Cause Race Condition — Duplicate Line Items

Attribute	Detail
Defect ID	QA-008
Severity	HIGH
Status	CLOSED
Module	Cart

Description

When a user rapidly tapped the Add to Cart button multiple times within a short interval (typically less than 300ms between taps), the application executed multiple concurrent Firestore write operations before any single write had completed and the UI had refreshed. This resulted in duplicate cart line items for the same product being created in Firestore, leading to incorrect quantity totals and subtotal calculations.

Steps to Reproduce

1. Navigate to any product detail screen. 2. Tap 'Add to Cart' five times rapidly in succession. 3. Navigate to Cart. 4. Observe: Product appears as 3–5 separate line items instead of one item with quantity 5.

Expected Behaviour

Concurrent add requests are serialised; the product appears as a single line item with the correct aggregate quantity.

Resolution Applied

Add to Cart button disabled after first tap until Firestore write completes. Request debounce of 500ms added.

QA-009 [MEDIUM] — Search Returns No Results for Partial Matches on Product Names

Attribute	Detail
Defect ID	QA-009
Severity	MEDIUM
Status	CLOSED
Module	Product Catalog

Description

The product search feature performed an exact-match Firestore query rather than a prefix or substring match. A search for 'ring' returned no results if the product name was 'Gold Diamond Ring', because the query used `whereEqualTo` rather than a range query or Firebase Full-Text Search. Only complete, exact product name matches returned results.

Steps to Reproduce

1. Navigate to search. 2. Enter 'ring'. 3. Observe: No results. 4. Enter 'Gold Diamond Ring'. 5. Observe: Correct results.

Expected Behaviour

Search returns all products whose names contain the search term as a substring.

Resolution Applied

Firestore query updated to use `.startsWith()` range query pattern (`orderBy + startAt + endAt`). For full substring support, Algolia integration is noted as Iteration 2 item.

QA-010 [MEDIUM] — Product Edit Form Does Not Pre-Populate Existing Values on Re-Open

Attribute	Detail
Defect ID	QA-010
Severity	MEDIUM
Status	CLOSED
Module	Admin — Products

Description

When an administrator opened the Edit Product form for an existing product, closed it without saving (using the back navigation), and then immediately reopened the same product's Edit form, all fields were displayed empty rather than pre-populated with the original product values. The product's data was only loaded once in `initState()` and the state was not reset on form re-open.

Steps to Reproduce

1. Open Edit Product for any product. 2. Note the pre-populated values. 3. Tap Back without saving. 4. Re-open Edit Product for the same product. 5. Observe: All fields now display as empty.

Expected Behaviour

Edit Product form always pre-populates with the current saved values for the product.

Resolution Applied

`TextEditingController` disposal issue resolved. Data loading moved to a `didChangeDependencies` override; controllers are rebuilt on each form open.

QA-011 [MEDIUM] — Application Displays Generic Error on Network Loss During Active Browsing

Attribute	Detail
Defect ID	QA-011
Severity	MEDIUM

Status	CLOSED
Module	Offline Behaviour

Description

When the device lost network connectivity during an active product catalog browsing session, the application displayed a generic unhandled exception dialog rather than a user-friendly offline indicator. The connectivity failure was not intercepted at the service layer, causing the StreamBuilder in the product listing screen to enter an error state and surface the raw exception.

Steps to Reproduce

1. Launch application and browse product catalog while connected. 2. Disable Wi-Fi and mobile data during browsing. 3. Scroll to trigger a new data load. 4. Observe: Generic error dialog appears with no actionable options.

Expected Behaviour

User-friendly offline banner displayed with 'Retry' option. Previously loaded products remain visible from cache.

Resolution Applied

NetworkStatusProvider added using connectivity_plus package. All StreamBuilders wrapped with connectivity-aware error states. Cached Firestore data displayed on connectivity loss.

QA-012 [MEDIUM] — Re-Sent OTP Does Not Update the Verification ID — Login Fails

Attribute	Detail
Defect ID	QA-012
Severity	MEDIUM
Status	CLOSED
Module	Authentication

Description

When a user requested a second OTP (after the first had expired or was not received), the application retained the verificationId from the first OTP request. The second OTP, associated with a new verificationId, would be rejected by Firebase if the user entered it, because the credential was built using the stale verificationId.

Steps to Reproduce

1. Enter phone number and request OTP. 2. Do not enter the code; wait for the initial OTP to expire, or tap Resend. 3. Enter the newly received OTP. 4. Observe: Login fails with 'Invalid verification code' error.

Expected Behaviour

Re-sent OTP is associated with a new verificationId that is stored and used for subsequent credential creation.

Resolution Applied

verificationId state variable updated in the OTP callback handler on each send request. Both the first and any subsequent OTP callbacks now correctly update the state.

8.13 Remaining Defects Summary (QA-013 to QA-043)

The following table provides a summary of the remaining 31 defects not documented in full detail above. All were resolved in the developer remediation cycle. Full defect records including screenshots and steps are available in the defect tracking system.

ID Range	Module	Severity	Count	Status
QA-013 – QA-016	Admin — Products	Medium	4	CLOSED
QA-017 – QA-020	Product Catalog	Medium	4	CLOSED
QA-021 – QA-022	Admin — Staging	Medium	2	CLOSED
QA-023 – QA-024	Push Notifications	Medium	2	CLOSED
QA-025 – QA-026	Admin — Banners	Medium	2	CLOSED
QA-027 – QA-030	Authentication	Low	4	CLOSED
QA-031 – QA-033	Product Catalog	Low	3	2 closed, 1 accepted
QA-034 – QA-035	Offline Behaviour	Low	2	1 closed, 1 accepted
QA-036	Cart	Low	1	Accepted (Known Limitation)
QA-037 – QA-040	Admin — Products	Cosmetic	4	CLOSED
QA-041 – QA-043	Various	Cosmetic	3	CLOSED

9. Regression Testing Results

A full regression pass was executed at the start of Cycle 2 using the automated integration test suite. The regression suite comprised 847 test cases covering all in-scope modules. All 847 tests executed against the Cycle 2 release build with zero failures and zero new defects introduced. No test cases required manual intervention during the automated run.

Metric	Value
Automated Test Cases Executed	847
Tests Passed	847
Tests Failed	0
Tests Skipped	0
New Defects Introduced	0
Regression Duration	12 minutes, 34 seconds (on CI runner)
Device Used	Samsung Galaxy S23 — Android 14
Build Tested	Release APK — Cycle 2

Twelve test cases from the original suite were updated during Cycle 2 to account for intentional UI changes made by the development team during remediation, specifically the OTP cooldown timer UI and the restructured error message display. These test updates did not alter the acceptance criteria; only the expected UI element identifiers were modified.

10. Risks and Quality Assessment

10.1 Quality Risk Assessment

Risk Area	Risk Level	Mitigation
Firestore Security Rules Not Deployed	HIGH	Deployment is a prerequisite for go-live. Engineering team has provided rules and deployment guide.
Limited Edge-Case Coverage for Admin Publishing Flow	MEDIUM	Additional exploratory testing planned for post-launch staging cycle.
Performance on Low-End Devices (API 21–23)	MEDIUM	Acceptable at current scope. Image compression already applied. Deferred optimisation to Phase 2.
Deep Link Routing on Android 10 vs 14 API Differences	LOW	Tested on both API levels. Behaviour confirmed consistent post-fix (QA-006).
Third-Party FCM Delivery Reliability	LOW	Firebase SLA 99.9%. Out of application's direct control. Documented in SLA.

10.2 Known Limitations (Accepted Defects)

ID	Description	Severity	Rationale for Acceptance
QA-033	Search does not	Low	Full-text search via

	support full substring match — prefix match only		Algolia deferred to Iteration 2. Current prefix match is functional.
QA-034	Offline cached data not available for the first application cold start	Low	Firebase Firestore offline persistence requires at least one successful sync. Acceptable operating condition.
QA-036	Cart quantity spinner does not auto-correct on invalid non-numeric free-text input on some keyboards	Low	Affects less than 5% of keyboard configurations. Numeric keyboard enforced on tested devices.

11. Recommendations and Next Steps

Deploy Firestore Security Rules — Immediate

This is the single highest-priority action prior to production launch. The deployment guide is included in the companion Deployment Guide document, and the rules themselves are reproduced in the VAPT Report Appendix C.

Expand Automated Test Coverage to 80% in Iteration 2

The current automation coverage of 68% is above the minimum 60% threshold. Targeting 80% coverage in Iteration 2 will reduce manual regression effort and provide earlier defect detection as new features are added.

Implement Full-Text Search for the Product Catalog (Iteration 2)

The current search functionality is limited to prefix matching due to Firestore query constraints. The integration of Algolia or a Firebase Extension for full-text search would significantly improve the product discovery experience.

Schedule a Formal Performance Audit at 1,000+ Product Scale

The current test environment held approximately 200 products. As the catalog grows to production scale (1,000+ products), Firestore pagination and query performance should be re-evaluated and index strategies reviewed.

Implement Continuous Integration QA Pipeline

Integrate the Flutter integration_test suite into the CI/CD pipeline (e.g., GitHub Actions or Firebase Test Lab) so that regression tests run automatically on every pull request, preventing the reintroduction of resolved defects.

12. Final Quality Status and Sign-Off

Based on the results of Cycle 1 testing, the subsequent developer remediation programme, and the Cycle 2 regression and retest pass, the QA Engineering Team hereby issues a formal quality assessment for the Vishal Gold Android application.

Acceptance Criterion	Target	Actual	Status
Cycle 2 Pass Rate	≥ 98%	99.1%	PASS
Open Critical Defects	0	0	PASS
Open High Defects	0	0	PASS
Open Medium Defects	0	0	PASS
Automation Coverage	≥ 60%	68%	PASS
Regression Tests Pass Rate	100%	100%	PASS
Performance: Cold Start (mid-range)	≤ 3.0s	2.4s	PASS
Performance: Screen Transition	≤ 300ms	190ms	PASS
Zero New Defects Introduced in Cycle 2	Yes	Yes	PASS

Overall Quality Verdict

QUALITY VERDICT: PASS with Conditions. The Vishal Gold Android application has satisfied all mandatory acceptance criteria. Zero Critical and High defects are open. Three Low-severity items are accepted as documented Known Limitations. The application is approved for production deployment, conditional on the deployment of Firestore security rules before public access is granted.

Authorization & Sign-Off

The undersigned parties confirm that they have reviewed the contents of this document and agree to its terms and findings.

Name: _____

Signature: _____

Date: _____

QA Lead | Engineering QA Team

Name: _____

Signature: _____

Date: _____

Engineering Lead | Development Team

Name: _____

Signature: _____

Date: _____

Project Manager | Delivery Team

Name: _____

Signature: _____

Date: _____

Client Acceptance | Vishal Gold

Appendix A — Test Data Reference

All test data used during this engagement was synthetically generated. No real customer data was used during testing. Test accounts were created specifically for this engagement and disabled post-testing.

Account Type	Purpose	Created By	Status
Retailer test account (x3)	Retailer role testing	QA Team	Disabled post-test

Wholesaler test account (x2)	Wholesaler role testing	QA Team	Disabled post-test
Administrator test account (x1)	Admin function testing	QA Team	Disabled post-test
OTP simulation SIM card (x2)	Authentication testing	QA Team	Revoked post-test

Appendix B — Tools and Frameworks

Tool	Version	Purpose
Flutter Integration Test	3.32	Automated functional and regression testing
Android Debug Bridge (ADB)	34.0	Device control, log capture
Firebase Test Lab	—	Cloud device testing (emulated devices)
Charles Proxy	4.6	Network traffic inspection and validation
Jira	Cloud	Defect tracking and sprint management
Confluence	Cloud	Test plan and test case documentation
TestRail	—	Test case management and execution tracking
Firebase Console	Web	Log inspection, Firestore data verification
Figma	—	UI reference for design compliance checks

Appendix C — Test Metrics Summary

Metric	Value
Total Test Cases	1,247
Total Automated	848
Total Manual	439
Cycle 1 — Executed	1,285
Cycle 1 — Passed	1,085 (87.0%)
Cycle 1 — Failed	162 (12.6%)
Cycle 1 — Blocked	38 (3.0%)
Cycle 2 — Executed	1,247
Cycle 2 — Passed	1,236 (99.1%)
Cycle 2 — Failed	11 (0.9%)
Cycle 2 — Blocked	0
Total Defects Raised	43
Defects Closed	40

Defects Accepted as Limitations	3
Regression Tests Passed	847 / 847 (100%)
Automation Coverage	68%
Cold Start Time (Samsung A52)	2.4 seconds
Average Screen Transition	190 ms
Total Testing Duration	8 days (Feb 17 – Feb 25, 2026)